

Pipes and Redirection

In this chapter, you’re going to learn how to harness one of the most powerful computing concepts in existence: pipes! Pipes can be used to connect commands, building up complex, customized flows that accomplish a specific task. By the end of the chapter, you’ll be able to understand (or compose) something like this:

```
history | awk '{print $2}' | sort | uniq -c | sort -rn | head -n 10
```

In case you’re curious, this prints out a top-10 list of your most commonly used shell commands; on my machine, it produces this output:

```
1000 git
 115 ls
 102 go
   83 gpo (an alias I've set up for pushing a local git branch to the origin)
   68 make
   65 cd
   59 docker
   42 vagrant
   35 G00S=linux
   30 echo
```

To really understand pipes, you need to first understand file descriptors and input/output redirection, so that’s where we’ll start. Some of the information in this chapter is quite dense; just take your time and try out all the examples to make sure you understand everything. The time you invest in learning these concepts now will save you many hours throughout your career.

In this chapter, we’ll cover the following topics:

- File descriptors
- Connecting commands together with pipes (|)
- The CLI tools you need to know
- Practical pipe patterns
- Inspecting file descriptors

File descriptors

You’re probably familiar with file handles (also known as *file descriptors*) from your software engineering experience. If not, we recommend you check out *Chapter 5, Introducing Files*. In short, if your program needs to read or write a file on the operating system, opening that file gives you a “file handle” to it – a pointer, or reference, to that file object.

Because the operating system mediates all access to system resources like files, it tracks which file handles, or descriptors, your program is actively referencing.

But even if a process doesn’t touch a single file on the operating system, it’s got some file handles open. In Unix-like operating systems, every process has at least three file descriptors:

- `stdin`: standard input - or, `fd 0` (“file descriptor zero”)
- `stdout`: standard output - or, `fd 1` (“file descriptor one”)
- `stderr`: standard error - or, `fd 2` (“file descriptor two”)

These first three file descriptors function as standard communication channels to (and from) a process. As a result, they exist in the same order for every process created on the system. The first always points to a file which will be used to read in input. The second points to a file that will be used for writing output. And the third references a file that will receive error output.

Optionally, after those first three standard file descriptors, there can be any number of other file descriptors/handles, based on what the program is doing. Your process could have:

- Files it’s working with
- Sockets it’s reading from or writing to (think Unix or TCP sockets being written to for networking)
- Devices like keyboards or disks it needs to use

What do these file descriptors reference?

You now know, from the perspective of a process, what these file descriptors are used for:

- `0` (`STDIN`): get input from here
- `1` (`STDOUT`): put regular output here
- `2` (`STDERR`): put error output here

But if we zoom outside of a single process, which files are these file descriptors actually pointing to? Where does input come from, and where do output and errors get written to?

Let’s use a Bash shell process as an example: by default, it takes input (`STDIN`) from your terminal (which is represented by a file on the filesystem). Bash prints output and errors to the same terminal. In essence, your entire shell session is happening via read and write operations to a single file. You’ll learn much more about Bash in the next chapter, *Automating Tasks with Shell Scripts*.

Let’s look at this kind of input and output redirection in more detail.

Input and output redirection (or, playing with file descriptors for fun and profit)

This knowledge comes in handy quite often during real-life development tasks: every time you want to avoid typing lots of input and take it from a file instead, or when you want to log the output of a program, and many more situations. When you create a process, you can control where its three standard file descriptors point, with powerful results.

Input redirection: `<`

The `<` (less-than) symbol lets you control where a process gets its input from. For example, you’re used to giving input to Bash with your keyboard, one command at a time. Let’s try giving Bash input from a file, instead!

Assume I have a file named `commands.txt` with the following content (I’m using `cat` here to print out my example file):

```
# cat commands.txt
pwd
echo "hello there, friends"
echo $SHELL
cd /tmp
pwd
```

These are valid shell commands, as far as Bash is concerned, so I’m going to launch a new Bash process and use this file as standard input:

```
# bash < commands.txt
/tmp/gopsinspect
hello there, friends
/bin/bash
/tmp
```

Instead of prompting me for input and waiting until I give it, Bash reads and executes one line at a time: it reads input from the file until it comes across a newline (`\n`) character, and just as if you’d hit the *RETURN* key, it executes the command.

In this example, the program’s standard output is still going back to our terminal, where we can read it. Let’s change that now.

Output redirection: `>`

We want to redirect `STDOUT` (file descriptor `1`) to a file instead of a terminal, logging the output of each command instead of printing it out to the terminal in real time:

```
# bash < commands.txt > output.log
```

Notice that there is no visible output in the terminal now – because the `>` character has redirected output to `output.log`. Use `cat` to print out the log file and confirm that it contains the expected output:

```
# cat output.log
/tmp/gopsinspect
hello there, friends
/bin/bash
/tmp
```

Interestingly, you’ll notice that because file descriptor `1` is standard output, writing `>` is the same as writing `1>`. You’ll rarely see a `1` used, because it’s assumed that standard output is being redirected. In other words:

```
date > mydate.log
is equivalent to writing
date 1> mydate.log
```

Use `>>` to append output without overwriting

In the previous example, we created a log file by redirecting command output with `>`. If you run the example a few times, you’ll notice that the log file doesn’t grow at all. Each time you redirect output to a file with `> filename`, anything in that file will be overwritten.

To avoid that – as in the case of a long-lived log file that collects output from more than a single process or command – use `>>` (append). This will simply append to your output file, instead of overwriting its entire contents each time.

We’ll cover Bash scripts in more detail in a later chapter, but for now, here’s a quick script that writes a timestamp of the current time to a log file once per second:

```
while true; do
    date >> /tmp/date.log
    sleep 1
done
```

In this example script, we create an infinite loop (`while true; do [ ... ] done`) which runs the `date` command. It redirects the output of this command to the `/tmp/date.log` file using `>>`, which appends the output to the file (`>` would overwrite the file each time). Then, the script sleeps for one second, and starts again from the beginning.

Running the `date` command once produces the following output:

```
→ ~ date
Sat Jan  6 16:39:37 EST 2024
```

Running this script, on the other hand, does nothing visible at first, because the output is being redirected to a file. Here’s what it looks like when I paste this little script into my terminal, let it run for a bit, kill it with `Ctrl + C`, and then print out the contents of the file it created:

```
→ ~ while true; do
    date >> /tmp/date.log
    sleep 1
done
^C%
→ ~ cat /tmp/date.log
Sat Jan  6 16:44:01 EST 2024
Sat Jan  6 16:44:02 EST 2024
Sat Jan  6 16:44:03 EST 2024
[ ... ]
Sat Jan  6 16:44:08 EST 2024
```

You’ll use this kind of simple output redirection in all kinds of everyday situations, like creating an ad hoc log file for a quick debug script you throw together.

Error redirection with `2>`

Many command-line programs that have a lot of expected output will also output occasional errors – think of a `find` command that encounters occasional “permission denied” errors for directories you’re not allowed to peek inside.

Although these kinds of errors are minor and expected, you don’t want them mixed in with everything else, polluting your output. This becomes especially important when you’re not using command-line tools interactively, but rather writing small scripts or larger programs that process the output of the commands you’re running.

You’ve seen how to redirect Standard Input (`fd 0`) and Standard Output (`fd 1`). Let’s look at how to redirect Standard Error (`fd 2`) using the `2>` (redirect file descriptor 2) syntax:

```
find /etc/ -name php.ini > /tmp/phpinis.log 2>/dev/null
```

This command searches for any files named `php.ini` inside the `/etc` directory tree. The files it finds (`find`'s `STDOUT`) are written to `/tmp/phpinis.log`, and any errors it encounters are ignored by sending them to a special file called `/dev/null`.



Tip

`/dev/null` is a special file-like object that returns zeros when you try to read from it and ignores anything written to it – it’s used as a kind of garbage dump for output that engineers want to silence or ignore. You’ll see it used quite often in scripts.

Now that you’ve seen input and output redirection, let’s look at pipes, which put both of those concepts together: they redirect the output of one command to the input of another.

Connecting commands together with pipes (|)

You’ve learned how to redirect each of the three standard file descriptors to various locations and seen why that’s often useful. But what if, instead of just redirecting input and output to and from various files, you wanted to connect *multiple programs* together?

On the command line, you can use the pipe character (`|`) to connect the output of one program to the input of another program. This is an extremely powerful paradigm that is heavily used in Unix and Linux to create custom sorting, filtering, and processing commands:

```
echo -e "some text \n treasure found \n some more text" | grep treasure
```

If you paste this into your shell, you’ll see `treasure found` printed out. Here’s what happened:

1. The first command, `echo`, runs and produces the output you see between double quotes (the newline characters make this a 3-line string).
2. The pipe character streams that output (file descriptor 1) to the input of the next command (file descriptor 0), `grep`. `grep`'s input is now hooked up to the output of the previous command.
3. The `grep` command looks at each newline-delimited line in turn and finds a match for `treasure` on the second line. `grep` prints that second line to its standard output.

Multi-pipe commands

Here’s the – fairly extreme – example you saw at the beginning of the chapter:

```
history | awk '{print $2}' | sort | uniq -c | sort -rn | head -n 10 > /tmp/top10commands
```

Each pipe in this complex command simply takes the output of the previous command (`STDOUT`) and uses it as the input (`STDIN`) for the next command.

Piping the output of one command into the input of another is what enables these kinds of flows, filtering and sorting the data streaming between these commands without actually having to write any custom software. Just because there’s no program called `top10commands` doesn’t mean you can’t quickly cobble one together with existing, standard commands like this.

Reading (and building) complex multi-pipe commands

No matter how complex or magical some of the piped-together commands you encounter will seem, they were all built the same way: one command at a time. Whether you’re trying to read a complex series of commands like this or creating one of your own, the process is the same:

1. Take the first command and make sure you understand what it does, at a basic level. Scan the man page or other documentation if you aren’t familiar with it.
2. Run the command and inspect its output.
3. Add the pipe and the command following it.
4. Repeat from *step 1* until you’ve made it all the way through the commands.

You’ll see that even the scariest shell/pipe monstrosities become manageable when you apply this process. Always remember that you’re just dealing with a data stream, which flows through the pipes from command to command, being shaped, modified, filtered, redirected, and transformed along the way.

We’ll discuss this more in *Chapter 12, Automating Tasks with Shell Scripts*, but try to be respectful of other programmers who must read your code: limit your statements to two or three pipes, and use well-named variables to store intermediate results for easy reading if your memory constraints allow it.

Now that you’ve seen how the primitives of file descriptors are exposed as easy-to-use input and output redirection, let’s look at some real-world examples of useful program combinations that rely on this composability that’s built into Unix.

The CLI tools you need to know

Before we jump into the kinds of wild combinations that you saw at the beginning of the chapter, let’s look at some of the most common Unix helper tools that are used to filter, sort, and glue together these data streams you’ll be creating on the command line.

cut

`cut` takes a delimiter (`-d`) and splits input on that delimiter, like `String.Split()` or `String.Fields()` in many programming languages. You then select which field (list element) you want to output with `-f`, for example, `f1` for the first field.

If you feed `cut` more than one line of input, it will repeat that same operation on all lines:

```
echo "this is a space-delimited line" | cut -d " " -f4
space-delimited
```

You can see how using different delimiters for cut would work, too; in the following example, we cut on the hyphen character instead of a space:

```
→ ~ echo "this is a space-delimited line" | cut -d "-" -f1
this is a space
→ ~ echo "this is a space-delimited line" | cut -d "-" -f2
delimited line
```

You can see that this changes the number of fields available as well – two in this case, since there’s only one hyphen in the text. Trying to print the fourth field with `-f4`, as in the previous example, will just give you an empty line.

To get friendly names for all users with root in their names on an macOS machine, you can use the following:

```
# grep root /etc/passwd | cut -d ":" -f5
System Administrator
System Services
CVMS Root
```

sort

`sort` does a per-line sorting, alphabetical or numeric.

Reverse-sorting with `-r` is often useful when dealing with numeric data (`-n`). You’ll often want `-rn` together (see Top X in the Practical pipe patterns section of this chapter).

The `-h` flag can be very useful for sorting by human-readable output of many other commands, like this:

```
# du -h | sort -rh
1.6M      .
1.3M      ./ .git
1.2M      ./ .git/objects
60K       ./ .git/hooks
28K       ./ .git/objects/d8
```

uniq

Removes duplicate lines. This command needs sorted data to work the way you expect, otherwise it only checks whether each line is a duplicate of the previous line:

```
# cat /tmp/uniq
one
two
one
one
one
seven
one
```

Default behavior; probably not what you want:

```
# uniq /tmp/uniq
one
two
one
seven
one
```

`uniq` skips occurrences when they follow each other but leaves them when they’re separated by other text. Now the same thing, with sorted data:

```
# sort /tmp/uniq | uniq
one
seven
two
```

Counting

`uniq` also has a useful “count” option, accessible with `-c`. The same caveat about sorted input is worth restating here – for example, a file with the following content:

```
arch
alpine
arch
arch
```

Will produce the following output when run through `uniq -c`:

```
$ uniq -c /tmp/sort1.txt
 1 arch
 1 alpine
 2 arch
```

This is not what most users expect: there are 3 occurrences of `arch` in the file, but `uniq` shows two separate counts for the same word. To get the behavior you expect (`uniq` should return output that doesn’t contain any duplicate lines), your input must be sorted.

This is annoying for beginners, but very much in line with the Unix philosophy: tools should be small and sharp and shouldn’t duplicate functionality from each other. If you write a sorting tool, it should only sort, and if you write a uniquifying tool, it is allowed to depend on sorting from another tool to ensure extremely conservative (and consistent) memory usage.

Here, we sort before using `uniq`, which gets us the output we expect:

```
$ sort /tmp/sort1.txt | uniq -c
 1 alpine
 3 arch
```

You’ll notice that this sorts in ascending order, which isn’t what you want for the top-X list of commands you saw at the beginning of the chapter. To solve this, we do a “reverse numeric” sort (`-rn`) of this numbered list (since each line now starts with a number, thanks to `uniq -c`, this is easy to do). Here’s an example of this in action, on a file with many more duplicates:

```
$ sort /tmp/sortme.txt | uniq -c | sort -rn
 6 ubuntu
 4 alpine
 3 gentoo
 2 yellow dog
 2 arch
 1 suse
 1 mandrake
```

WC

With this command you can measure the word, line, character, and byte input counts. You can also count space-delimited words with `-w`:

```
# echo "foo bar baz" | wc -w
3
```

Line-counting is extremely common in the following format:

```
# wc -l < /etc/passwd
123
```

head

Head returns the first lines of a stream or file – 10 lines by default. Specify how many lines you want with **-n**:

```
# head -n 2 /etc/passwd
##
# User Database
```

tail

This is the opposite of **head**: it returns lines from the end of the file or stream. It takes **-n** just like **head**.

tail can also be used interactively for following along with a log file, even as that file has new data streamed/written to it. You’ll see it used a lot like this during troubleshooting:

```
tail -f /var/log/nginx/access.log
```

tee

Sometimes, one copy of the data from standard input just isn’t enough. **tee** copies standard input to standard output, while also making a copy in a file. As a software developer, I really like **tee** for two specific cases.

First, for debugging and logging: when I’m running scripts or programs that generate output, **tee** can be used to both display the output on the screen and log it to a file for later analysis. We’re using the **echo** command here, but you’d likely be calling your own program before the first pipe here:

```
# echo "Hello" | tee /tmp/greetings.txt
Hello
# cat /tmp/greetings.txt
Hello
```

The second use case where **tee** comes in handy is for copying data from a pipeline like the ones we’re learning to construct in this chapter. You can use **tee** to intercept this flow at any point in the pipe, and save/inspect the intermediate results without disrupting the pipeline.

Here’s the “top 10 commands” example from earlier, but with **tee** inserted before limiting the results to just 10. This saves the full results in a temp file before we truncate them:

```
history | awk '{print $2}' | sort | uniq -c | sort -rn | tee /tmp/all_commands.txt | head -n 10
```

Now if you want to see all of the commands, not just the top 10, you can just use **cat** or **less** to inspect the **/tmp/all_commands.txt** file.

awk

awk is often just used for dealing with columns of data, but it’s actually a whole language.

For example, you can grab the second column from each line in the following way:

```
# echo "two columns" | awk '{print $2}'
columns
```

sed

`sed` is a stream editor with tons of options. Most commonly, it’s used for character replacement in streams or files.

Imagine we have a file like this:

```
# cat /tmp/sensitive.txt
Nopasswords
not_a_password_either
sillypasswordtimes
password
ok this works
```

If we want to redact ONLY the line that has `password`, and nothing else, on it:

```
sed 's/^password$/REDACTED/' /tmp/sensitive.txt
nopasswords
not_a_password_either
sillypasswordtimes
REDACTED
ok this works
```

This example uses a file rather than an input stream coming from another command. By default, this will not modify the original file. If you *do* want to modify the input file, use the `-i` (in-place) option.

Now that you’ve been introduced to pipes and have seen some of the most common command-line tools, let’s put those building blocks together and learn several practical patterns that you can use to make your daily command-line life a bit easier.

Practical pipe patterns

As mentioned before, longer multi-pipe commands are built iteratively – one command at a time. However, there are some useful patterns that you’ll see re-used frequently.

“Top X”, with count

This pattern sorts the input by the number of occurrences, in descending order. You saw this in the original example from this chapter, which displayed the most frequently used shell commands from Bash’s history file.

Here’s the pattern:

```
some_input | sort | uniq -c | sort -rn | head -n 3
```

We can note the following details about this pattern:

- The input is sorted alphabetically, and then run through `uniq -c`, which needs sorted input to work on.
- `uniq -c` eliminates duplicates, but adds a count (`-c`) of how many duplicates it found for each entry.
- `sort` is run again, this time as a reverse-numeric (`-r` and `-n`) sort which sorts the unique counts from the input and outputs the lines in reverse (highest number first) sorted order.
- `head` takes that top ranking and cuts it down to three lines (`-n 3`), giving you the top three strings from the original input, along with how often they occurred.

This pattern can come in handy when you need to know the most common browser user-agents hitting your website, the IP addresses of the worst offenders who are trying to probe and exploit your website, or any other situation where a sorted, ranked list is useful.

curl | bash

The `curl | bash` pattern is a common shortcut used in Linux to download and execute scripts directly from the internet. This method combines two powerful command-line tools: `curl`, which fetches the content from a URL, and `bash`, the shell interpreter, which executes the downloaded script. This pattern is a significant time-saver, allowing developers to quickly deploy applications or run scripts without manually downloading and then executing them.

As an example, let’s install the Pi-hole adblocking DNS server using this pattern:

```
curl -sSL https://install.pi-hole.net | bash
```

Let’s break this down, step by step:

1. `curl -sSL https://install.pi-hole.net`: This fetches the Pi-hole installation script, which is hosted at this URL. We pass two options:

- `-sS`: Silent mode gives you the raw response from the server, but shows errors should they occur.
 - `-L`: Follow redirects.
2. `|`: The pipe symbol passes the output of the previous command (`curl`) as input to the next command (`bash`).
 3. `bash`: Executes the script fetched by `curl`.

This is a tremendously useful pattern for automating things like code deployments or local environment installation/configuration. However, take special care that the script you download and execute is not malicious. Blindly running scripts from the internet is extremely bad practice.

Security considerations for curl | sudo | bash

Anytime you trust a third party to run code on your machine, you’re trading some security for convenience. In that sense, using `curl | sudo | bash` to install something via a script hosted on a trusted server is not much different from using a package manager. Most package managers (except for `nix`) don’t have a particularly impressive security design either, but they generally give you a reasonable set of security features. You’re giving all of these security features up when you `curl | sudo | bash` an install script:

- There’s no package that can be checksummed and cryptographically signed to make sure you got the correct and official version.
- There’s no restriction on – or enforcement of – which servers you download from, and you don’t know how secure those servers are: you have no way to identify a compromised server hosting malicious install scripts.
- The scripts themselves are just code being run as the root user on your machine, so they can do anything *you* can on your machine, for better or worse. To be fair, many popular package managers also have this problem.

For all of these reasons, please heed our warning to split `curl` into its own step and read through the downloaded install script before running `sudo bash` to execute it. The main things to look out for are:

- Ensure the server/domain you’re downloading the script from is trustworthy; this should be a reputable developer’s website or a trusted third-party code hosting platform.
- Ensure you use HTTPS for the `curl` download (i.e., the URL should start with `https://`).
- Read through the script carefully, to see which commands it runs and where it pulls additional code or executables from. If it downloads additional scripts or executables, have a look at those too.

I think we’ve established that `curl | sudo | bash` is not a particularly secure method of installing software. Following these guidelines can help you be a bit safer if you – like most of us – give into temptation one day and follow this installation method for a specific piece of software (for example, `homebrew` on macOS).

Let’s look at another common pattern now: filtering and searching with `grep`.

Filtering and searching with grep

When you run commands that produce a lot of output, it’s generally best practice to filter the output down to just what you need. The most common tool for this is called `grep`, and you can think of it as a highly configurable text search or string-matching function. Here’s an example of what filtering might look like.

Imagine that you need to find a Linux process’s working directory. The `lsdf` tool can accomplish this:

```
→ ~ lsdf -p 3243 | grep cwd
vagrant 3243 dcohen  cwd    DIR          1,4        192        51689680 /Users/c
```

Here’s a quick description of what’s happening:

1. I’m getting a listing of open file handles for a specific process (PID 3243), using `lsdf`.
2. I’m then passing the results (`|`) to the `grep` utility and using that to search the results for the string `cwd`. There’s only one line of results that contains the string `cwd`, so that’s the only line that `grep` prints to the terminal.

This pattern is useful anytime you have a *lot* of data as input, but you only need a subset of that data that can be identified by a specific string. `grep` operates on lines of input text, so it’s hugely helpful for picking out data like:

- Loglines containing the IP address you are following
- Occurrences of a username in a piped data stream
- Lines that match a pattern (`grep` is regular-expression aware and can take string patterns in addition to literal search strings)

`grep` is a large and powerful tool that you’ll have occasion to use almost every day. For more information, check out the manpage for `grep` by typing `man grep`.

You’ve already seen `grep` used on files in this book (for example, `grep searchstring hello.txt`), but it’s also an invaluable filtering component in piped commands. Let’s look at a practical example now.

grep and tail for log monitoring

When you’re looking at production logs to try to figure out what’s wrong, you’ll often only want to see loglines containing certain keywords or search strings. To do that, you’d run something like:

```
tail -f /var/log/webapp/too_many_logs.log | grep "yourSearchRegex"
```

This pattern continuously monitors the log file for new entries whose content matches “yourSearchRegex”, so you can see only the logs you need for the task at hand.

find and xargs for bulk file operations

`xargs` is a powerful utility that gives you the power of iteration (in other words, a “for” loop) inside of a single command. By default, `xargs` takes each (space, tab, newline, and end-of-file delimited) chunk of input it receives and executes the specified program using that chunk as input. For example, if you need to search for specific file content across `ONLY` the files returned by a certain `find` query, you can run this command:

```
find . -type f -name "*.txt" | xargs grep "search_term"
```

This command finds all files whose names end with `.txt` and then uses `xargs` to apply the `grep` command to each file individually. This pattern is handy for searching or modifying multiple files at once. Please be forewarned that `xargs` is a powerful – and *large* – program, capable of doing many things (including string interpolation into the command it executes). We can’t cover it all here, so please read the manpage and scour the internet for examples if you’re in a situation where this kind of functionality would save the day.

sort, uniq, and reverse numerical sort for data analysis

This is a useful pattern that you saw applied at the beginning of the chapter, where I used it to filter a large command history to get a list of the “top X most popular commands run on this system.” The core pattern is this:

```
(input stream) | sort | uniq -c | sort -rn
```

Useful for analyzing data, this pattern sorts the data from the input stream, deduplicates it while counting unique occurrences, and then performs a reverse numerical sort to give you the deduplicated data, with the most common lines first.

This is commonly truncated with `| head -n $NUMBER` to get only the top `$NUMBER` of results:

```
history | awk '{print $2}' | sort | uniq -c | sort -rn | head -n 10
```

Here, we use `history` to fetch the entire shell command history. This gives us a series of lines like:

```
12 brew install --cask emacs
```

We’re only interested in the top-level command (in this case, `brew`), so we use `awk` to fetch the second column.

Then we sort so that duplicates of the same command occur next to each other in the stream. Then we remove those duplicates with `uniq`, adding a count of occurrences to each remaining one. Now we sort again – this time using `-rn` for a reverse numerical sort, which gives us the “top X” effect. Finally, we take the first 10 lines with `head`.

This prints out the aforementioned top-10 list of your most-used shell commands; on my machine, it produces this:

```
1000 git
115 ls
102 go
83 gpo (an alias I've set up for pushing a local git branch to the origin)
68 make
65 cd
59 docker
42 vagrant
35 GOOS=linux
30 echo
```

awk and sort for reformatting data and field-based processing

`awk` is more than a program; it’s a stream-processing language. If you work with data streams on Unix system, then spending a few days learning the basics can save you weeks of time over your career. That said, just using `$#` syntax to reference whitespace-delimited columns in each line of the data stream is a good start.

Let’s look at an example given a data stream like the following:

```
Foo bar baz
Some data is nice
```

When the `awk` interpreter sees `$1`, it interprets this to mean “the first column” or in this case `Foo` in line 1 and `Some` in line 2. `$2` is the second column (`bar`, `data`), and so on. This is an incredibly common feature to use when working with data that’s just a bit too complex for simple `cut` commands:

```
cat file.txt | awk '{print $2, $1}'
```

This would produce output like:

```
bar Foo
data Some
```

In this case, it prints out column 2 before column 1 for each file, and ignores all other data in each line. This is often used for reformatting and organizing data based on specific fields.

sed and tee for editing and backup

`sed` stands for **Stream Editor** and is used when you want to transform a data stream. You do this ten times a day in your text editor when you find/replace a symbol. The following command is essentially the command-line version of that functionality: it transforms all occurrences of `old` in `file.txt` into the string `new` and writes the resulting stream to a new file, `file.txt.changed`. It does this without making changes to the original `file.txt` file:

```
sed 's/old/new/g' file.txt | tee file.txt.changed
```

Although editing file content is an easy demonstration of this concept, `sed` is tremendously useful for transforming stream data as it zips from the output of one command to the input of the next:

```
(input stream) | sed 's/old/new/g' | (next command)
```

ps, grep, awk, xargs, and kill for process management

Although `pgrep` is a good utility for sending signals to all processes whose name matches a pattern, sometimes it’s just not available on your system. You can cobble together similar functionality (and get much more specific with what you want to target, not just the name) by using this set of piped-together commands:

```
ps aux | grep "process_name" | awk '{print $2}' | xargs kill
```

`ps` starts you off with a list of running processes, which `grep` filters to just those containing the pattern you’re searching for. `awk` gets the second column (the process ID) for each matching line, and then feeds all matched lines to `xargs` (our quasi for loop), which executes `kill` on each PID. This sends a `SIGTERM` to each matching process and (hopefully) halts it.

tar and gzip for backup and compression

Although many utilities have flags that let you do both, chaining together archiving and compression is another use case that makes sense. This gives you the added flexibility of adding additional chained commands. For example, if you want to add encryption, that’s just a single additional piped command away:

```
tar cvf - /path/to/directory | gzip > backup.tar.gz
```

This creates a compressed archive of a directory, commonly used for file backup and storage. You can see larger commands using this kind of pattern:

```
ssh user@mysql-server "mysqldump --add-drop-table database_name | gzip -9c" | gzip -d |
```

This is an especially fun example that logs into a database server using SSH, dumps out a database, compresses that data stream, shuttles it back to the local machine over SSH, decompresses it again, and finally dumps it into the local MySQL server.

Your aim shouldn’t necessarily be to write commands as complex as this one (or some of the others you’ve seen here), but if you know how to put something like this together in a pinch, it can get you out of some extremely tight spots as a developer. We hope this section has demonstrated that understanding the input and output redirection primitives that Unix systems expose to you – via `<`, `>`, `>>`, `|`, and file descriptors in general – is basically a superpower. Use it wisely.

Advanced: inspecting file descriptors

On Linux, you can easily see where a process’s file descriptors are pointing. We’re going to use the slightly magical `/proc` virtual filesystem to do just that.

Procs (the `proc` virtual filesystem) is a Linux-only abstraction that represents kernel and process state as files. The data inside of these files comes straight from the operating system kernel, and only exists while you’re reading them. Just listing the `/proc` directory will show you many files; here’s a selection of some of the more important ones, taken from the Arch Linux wiki:

```
/proc/cpuinfo - information about CPU
/proc/meminfo - information about the physical memory
/proc/vmstats - information about the virtual memory
/proc/mounts - information about the mounts
/proc/filesystems - information about filesystems that have been compiled into the kernel
/proc/uptime - current system uptime
/proc/cmdline - kernel command line
```

What’s most interesting to us with regard to file descriptors is something not shown in the listing above: `/proc` contains a directory for every single process running on the machine, named after each **process ID (PID)**.

In a process’s `/proc` directory, that process’s file descriptors are represented as symbolic links in a directory called `fd`. When you do a long listing on this `/proc/$PID/fd` directory, you’ll see that `l` is the first character in the long listing, which denotes a special `link` file, as you’ll recall from *Chapter 5, Introducing Files*.

Practically speaking, `/proc/1/` is the `init` process’s proc directory, and you can view init’s file descriptors by doing a long listing on `/proc/1/fd`.

Let’s look at the file descriptors for an interactive Bash shell process running on my machine, which `ps aux | grep bash` tells me is PID 9:

```
root@server:/# ls -alh /proc/9/fd
total 0
dr-x----- 2 root root  0 Sep  1 19:16 .
dr-xr-xr-x 9 root root  0 Sep  1 19:16 ..
lrwx----- 1 root root 64 Sep  1 19:16 0 -> /dev/pts/1
lrwx----- 1 root root 64 Sep  1 19:16 1 -> /dev/pts/1
lrwx----- 1 root root 64 Sep  1 19:16 2 -> /dev/pts/1
lrwx----- 1 root root 64 Sep  5 00:46 255 -> /dev/pts/1
```

You’ll notice that it’s an interactive shell session: its standard input is coming from a virtual terminal (`/dev/pts/1`), and its standard error and output are going back to that same terminal. That checks out.

Let’s look at a text editor like vim, which behaves a lot like a terminal – input and output happen via a terminal. However, there’s an added complication, which is that text editors usually keep one or more files open for writing. What does that look like?

In this example, I’m running the vim text editor, and editing a file in the `/tmp` directory. Let’s find the process ID for vim, so we know which `/proc` directory to look inside:

```
root@server:/# ps aux | grep vim
root      453  0.0  0.1 17232  9216 pts/1    S+   15:57   0:00 vim /tmp/hello.txt
root      458  0.0  0.0  2884  1536 pts/0    S+   15:58   0:00 grep --color=auto vim
```

There it is; process 453. Don’t be misled by the `grep` command which also includes `vim` in its command arguments. Now that we have the PID, let’s look at vim’s file descriptors:

```
root@server:/# ls -l /proc/453/fd
total 0
```

```
lrwx----- 1 root root 64 Jan  7 15:58 0 -> /dev/pts/1
lrwx----- 1 root root 64 Jan  7 15:58 1 -> /dev/pts/1
lrwx----- 1 root root 64 Jan  7 15:58 2 -> /dev/pts/1
lrwx----- 1 root root 64 Jan  7 15:58 3 -> /tmp/.hello.txt.swp
```

We can see that stdin (**0**), stdout (**1**), and stderr (**2**) are all pointing to a terminal device, just like a shell. And we also see that the editor has a file open, with file descriptor **3** linked to the file that vim is editing. When a process opens additional files, new file descriptors are created, and you can view them here.

Beyond being interesting for its own sake, this can come in handy when programs are behaving erratically due to bugs, or when you’re trying to trace what a potentially malicious program is doing. **procfs** is quite interesting and useful if you invest a bit of time in learning it: just type **man proc** to get started, or read the Arch Linux Wiki page for a gentler introduction at <https://wiki.archlinux.org/title/Procfs>.

Conclusion

In this chapter, we put together all of the previous skills and theory we’ve covered to unlock one of the most powerful features of Unix and Linux systems: streaming data through multiple commands using pipes and input/output redirection.

We started by showing you how the operating system exposes primitives like file descriptors, and then started looking at practical uses of input and output redirection. Then, we covered pipes, which are arguably one of the most useful features of Linux and other Unix operating systems. After covering the necessary theory and showing you some useful examples, we dove deep into the most common helper tools that people use to slice and dice the data streams that they build up using pipes. Finally, we showed you some of the most common and useful patterns and program-combinations that people use in the real world.

What’s in this chapter is the foundation for much of the advanced command-line usage you’ll encounter and use in your day-to-day work. You’ve now been exposed to some of the basic theory, tools, and patterns that you’ll see in the wild, which will make it easy to dive in and start building custom commands for common development, troubleshooting, and automation use cases.

To grow your skills, use what you’ve seen in this chapter in your day-to-day work! Use it as a reference for patterns to try, and keep learning new tools and commands that you can add to your own custom recipes and use to filter or otherwise manipulate data on the command line. You’ll feel like a wizard in no time.

Learn more on Discord

To join the Discord community for this book – where you can share feedback, ask questions to the author, and learn about new releases – follow the QR code below:

<https://packt.link/SecNet>

